PBA: 7. Prediction and Iteration

Fall 2026

Jaewon (“Jay-one”) Yoo

National Tsing Hua University

Where are we? Where are we going?

- **Where we have been**
  - Week 1: R and RStudio, functions and packages, Quarto
  - Week 2: data visualization. A table became a graph, layer by layer, with `ggplot2`
  - Week 3: data wrangling with `dplyr`. Rows, columns, and groups, then barplots from the summaries
  - Week 4: causality. Causal effects, experiments, and observational studies
  - Week 6: causality II. DAGs and which variables to control for
  - Week 7: relationships and tidying. Correlation, our own functions, joins

Where are we? Where are we going?

- **Where we have been**
  - Week 1: R and RStudio, functions and packages, Quarto
  - Week 2: data visualization. A table became a graph, layer by layer, with `ggplot2`
  - Week 3: data wrangling with `dplyr`. Rows, columns, and groups, then barplots from the summaries
  - Week 4: causality. Causal effects, experiments, and observational studies
  - Week 6: causality II. DAGs and which variables to control for
  - Week 7: relationships and tidying. Correlation, our own functions, joins
- **Today: prediction and iteration**
  - Predicting the 2020 election from state polls; evaluating the predictions
  - For loops: repeat one task over columns, states, and dates
  - Time-series plots of national polls with moving averages
  - After the lecture: today's code, then E4 at your own pace with the TA

Where are we? Where are we going?

- **Where we have been**
  - Week 1: R and RStudio, functions and packages, Quarto
  - Week 2: data visualization. A table became a graph, layer by layer, with `ggplot2`
  - Week 3: data wrangling with `dplyr`. Rows, columns, and groups, then barplots from the summaries
  - Week 4: causality. Causal effects, experiments, and observational studies
  - Week 6: causality II. DAGs and which variables to control for
  - Week 7: relationships and tidying. Correlation, our own functions, joins
- **Today: prediction and iteration**
  - Predicting the 2020 election from state polls; evaluating the predictions
  - For loops: repeat one task over columns, states, and dates
  - Time-series plots of national polls with moving averages
  - After the lecture: today's code, then E4 at your own pace with the TA
- **Where we are going**
  - Next class (Nov 5): regression and model fit. Then uncertainty

Roadmap

1. Prediction
2. Loops
3. Evaluating the Predictions
4. Time-Series Plot
5. Review: Observational Studies
6. In-Class Exercise E4

1/ Prediction

2016 US Presidential Election

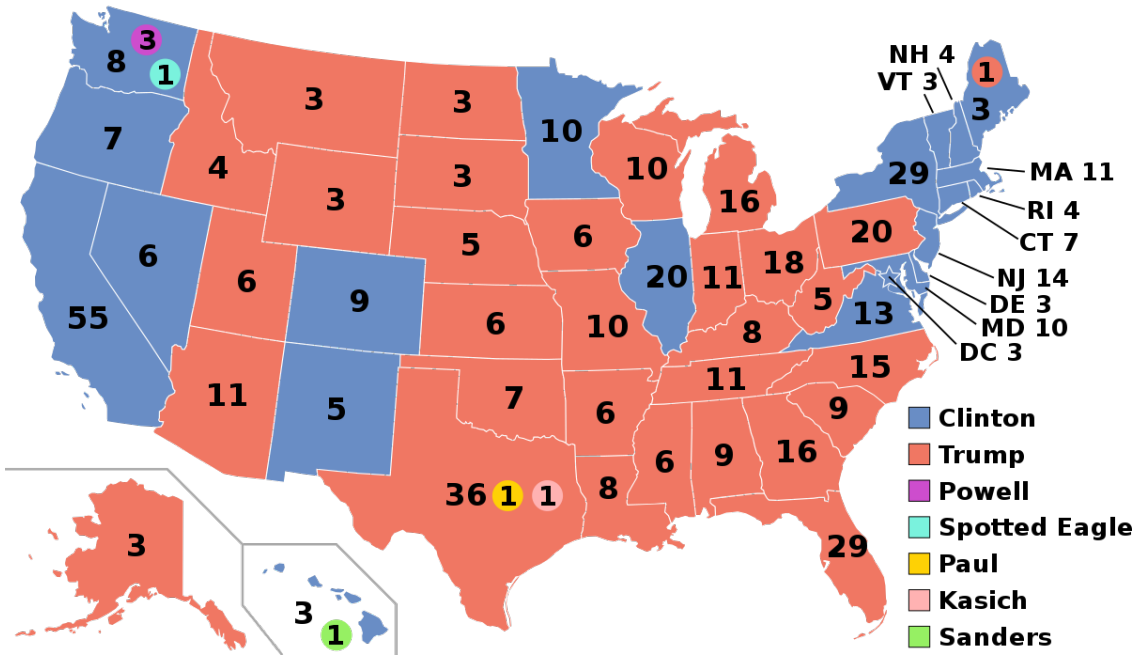
- Why? ⇝ Huge business impact with managerial/socio-economic implications (e.g., support for minimum wage policy)



Source: `Where They Stand on Economic Issues` by WSJ

- 2016 election popular vote:
  - H. Clinton: 65,853,514 (48.2%)
  - D. Trump: 62,984,828 (46.1%)
- Why did Trump win? **Electoral college!**
  - Trump = 304 pt; Clinton = 227 pt
- Election determined by 77,744 votes (margins in WI, MI, and PA)
  - 0.057% of all votes cast for president (~136.7 million)

Predicting Presidential Election Outcomes



Source: `ElectoralCollege2016.svg` by Gage (Wikimedia Commons, CC BY-SA 4.0)

- **Electoral college system**:
  - Must win an absolute majority of 538 electoral votes.
  - 538 = 435 (House of Representatives) + 100 (Senators) + 3 (DC)
  - Must win at least 270 votes
  - Nobody wins an absolute majority ⇝ House vote
- Implication: must predict winner of each state!

Our First Prediction Model

- A prediction model turns information we have now into a guess about an outcome we do not know yet.
    - Input: the latest polls in each state, taken before election day.
    - Rule: average the polls.
    - Output: the predicted margin for Biden in each state, and so the winner of each state.
- We check the model against an outcome that came later: the actual election results.
    - The model never saw the results, so this is an out-of-sample test.
- This is a **baseline**: the simplest model to start from. We build regression models in Weeks 9 and 10.

Prediction Strategy

- Predict state-level support for each candidate using polls/surveys
- Allocate electoral college votes of that state to its predicted winner
- Aggregate EC votes across states to determine the predicted winner

- Coding strategy:
  1. For each state, subset to polls within that state.
  2. Further subset the latest polls (i.e., latest information available).
  3. Average the latest polls to estimate support for each candidate.
  4. Allocate the electoral votes to the candidate who has the greatest support.
  5. Repeat this for all states and aggregate the EC votes.

$\rightsquigarrow$ Sounds like a lot of subsets!... What to do?

2/ Loops

A Simple Example

Cooperative Election Study (CES, formerly CCES; see Wikipedia or https://cces.gov.harvard.edu/)

- Q: What if we wanted to know the number of unique values of each column of the `cces_2020` data?

```
# today's five data sets are also in the course package pbadata (see the top of the R script)
# f=factor; d=double/numeric
> cces_2020 <- read_csv("https://bit.ly/49AZaAM", col_types = "fffdf"); cces_2020

# A tibble: 51,551 × 6
   gender race  educ                pid3        turnout_self pres_vote
   <fct>  <fct> <fct>               <fct>              <dbl> <fct>
 1 Male   White 2-year              Republican             1 Donald J. Trump (Republican)
 2 Female White Post-grad           Democrat              NA NA
 3 Female White 4-year              Independent            1 Joe Biden (Democrat)
 4 Female White 4-year              Democrat               1 Joe Biden (Democrat)
 5 Male   White 4-year              Independent            1 Other
 6 Male   White Some college        Republican             1 Donald J. Trump (Republican)
 7 Male   Black Some college        Not sure              NA NA
 8 Female White Some college        Independent            1 Donald J. Trump (Republican)
 9 Female White High school graduate Republican            1 Donald J. Trump (Republican)
10 Female White 4-year              Democrat               1 Joe Biden (Democrat)
# ℹ 51,541 more rows
# ℹ Use `print(n = ...)` to see more rows
```

Manually Changing Values

- Manually 'iterate' through each column:

```
> length(unique(cces_2020$gender))
[1] 2
> length(unique(cces_2020$race))
[1] 8
> length(unique(cces_2020$educ))
[1] 6

# Repeat for the rest of the columns, i.e., pid3, turnout_self, pres_vote.
```

- Subsetting with brackets: recall we can also access variables with [[]].

```
> unique(cces_2020$gender) # Point to the first column, gender
[1] Male   Female
Levels: Male Female
> unique(cces_2020[[1]])
[1] Male   Female
Levels: Male Female

> unique(cces_2020$pid3) # Point to the 4th column, pid3
[1] Republican  Democrat    Independent Not sure    Other
Levels: Republican Democrat Independent Not sure Other
> unique(cces_2020[[4]])
[1] Republican  Democrat    Independent Not sure    Other
Levels: Republican Democrat Independent Not sure Other
```

Manually Changing Values, Alternative

```
> length(unique(cces_2020[[1]])) # No. unique values for 1st col.
[1] 2
```

```
> length(unique(cces_2020[[2]])) # No. unique values for 2nd col.
[1] 8
```

⋮

```
> length(unique(cces_2020[[5]])) # No. unique values for (k-1)th col.
[1] 3
```

```
> length(unique(cces_2020[[6]])) # No. unique values for kth col.
[1] 7
```

Recognizing the 'Template'

What if you had more values? ⇝ Not efficient!

Recognize the template/recurring pattern:

```
length(unique(cces_2020[[<<column number>>]]))
```

Can we give R this template and a set of column numbers, and have it do our task repeatedly?

Loops in R

A **for loop** provides a way to execute these templates multiple times:

```
> output <- rep(NA, times = ncol(cces_2020))      # 1. output
> for (i in seq_along(cces_2020)) {               # 2. sequence
    output[i] <- length(unique(cces_2020[[i]]))   # 3. body
  }

> output
[1] 2 8 6 5 3 7
```

- Elements of a loop:
  1. `output`: vector to hold the output.
  2. `i`: placeholder name we'll use to swap values between iterations.
  3. `seq_along(cces_2020)`: vec. of values for the placeholder to take.
  4. `body`: a set of expressions that will be repeatedly evaluated.
  5. `{}`: curly braces to define beginning and end of the loop.
- Indentation is important for readability of the code.

- Recall: predict winner of each state to predict election outcome.
  - What to do? ⇝ State-level polling data

Watching the Loop Run

- Add `print(output)` inside the loop to see each iteration:

```
> output <- rep(NA, times = ncol(cces_2020))
> for (i in seq_along(cces_2020)) {
    output[i] <- length(unique(cces_2020[[i]]))
    print(output)
  }
[1]  2 NA NA NA NA NA
[1]  2  8 NA NA NA NA
[1]  2  8  6 NA NA NA
[1]  2  8  6  5 NA NA
[1]  2  8  6  5  3 NA
[1] 2 8 6 5 3 7
```

- Each iteration fills one more element of `output`.
- When `i` is 1, R works on the 1st column (`gender`); when `i` is 2, on the 2nd column (`race`); and so on.

The Same Task with `across()`

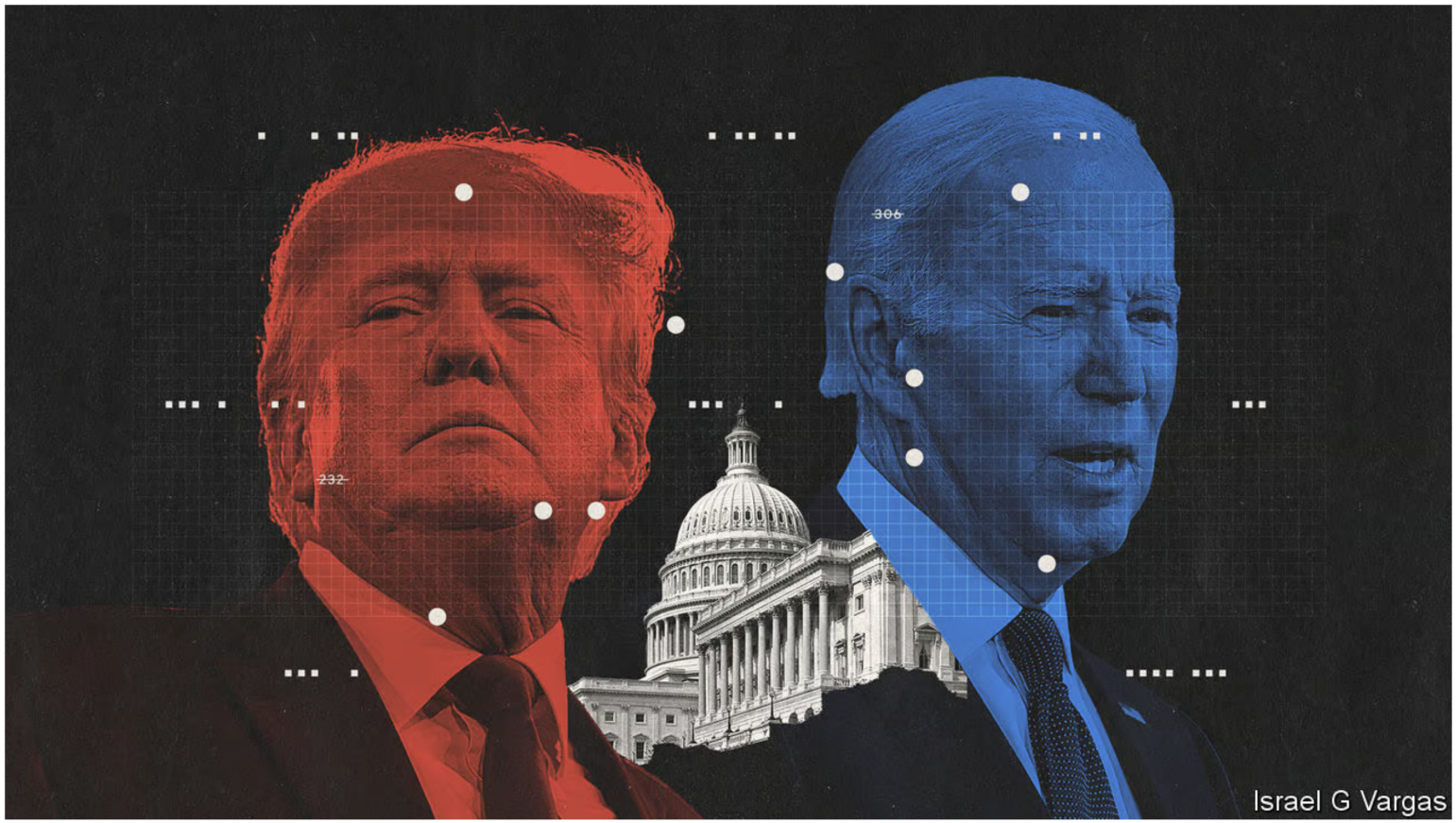
- Last week, `across()` applied our own `z_score()` to many columns.
- `across()` can apply other functions too. `n_distinct()` counts the unique values, like `length(unique())`:

```
> cces_2020 |>
    summarize(across(everything(), n_distinct))

# A tibble: 1 × 6
  gender  race  educ  pid3 turnout_self pres_vote
   <int> <int> <int> <int>        <int>     <int>
1      2     8     6     5            3         7
```

- We get the same numbers as with the loop, without writing the loop ourselves.

2020 US Presidential Election



Source: Donald Trump and Joe Biden both want to run again by The Economist

2020 Polling Prediction

- Election data: `pres20` (MIT Election Lab; electoral votes: National Archives)

Name	Description
`state`	abbreviated name of state
`biden`	Biden's vote share (percentage)
`trump`	Trump's vote share (percentage)
`ev`	number of electoral college votes for the state

- Polling data: `polls20` (FiveThirtyEight)

Name	Description
`state`	state in which poll was conducted
`end_date`	end date of the period when the poll was conducted
`days_left`	number of days between end date and election day
`pollster`	name of organization conducting poll
`sample_size`	number of people surveyed in the poll
`biden`	predicted support for Biden (percentage)
`trump`	predicted support for Trump (percentage)

What Is a Poll?

- A poll asks a sample of people (often likely voters) whom they will vote for.
  - It measures vote intention before the election. It is not the vote itself.

- A poll can miss the result in two ways:
  - **Sampling error**: a different sample gives a different number. Larger samples and more polls make it smaller.
  - **Systematic error**: the people who answer differ from the people who vote (for example, some groups answer polls less often). Averaging more polls does not remove it if the polls share the same error.

- Which kind of error did the 2020 polls have? We check on slide 25. We study samples in Week 11.

Import and Examine Data

- We will use `polls20` to predict the outcome.
  - Calculate Biden's margin of victory:

```
> polls20 <- as_tibble(read.csv(url("https://bit.ly/3R1qKB8")))
> polls20 <- polls20 |>
    mutate(margin = biden - trump)
```

- Use `glimpse()` to take a look at the data:

```
> glimpse(polls20)
Rows: 2,445
Columns: 8
$ end_date    <chr> "2020-11-02", "2020-11-02", "2020-11-02", "2020-11-02", "2020-11-02"...
$ state       <chr> "FL", "PA", "FL", "FL", "NV", "GA", "SC", "MT", "ME", "AZ", "FL", ...
$ days_left   <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,...
$ pollster    <chr> "The Political Matrix/The Listener Group", "Susquehanna", ...
$ sample_size <int> 966, 499, 400, 1054, 1024, 1041, 817, 920, 1024, 610, 1261, 690, 110...
$ biden       <dbl> 44.2, 48.4, 47.0, 47.3, 48.4, 45.4, 39.0, 45.0, 52.0, 50.0, 53.0, 52...
$ trump       <dbl> 48.0, 49.2, 48.2, 49.4, 49.1, 49.7, 51.4, 50.0, 40.0, 47.5, 45.5, 46...
$ margin      <dbl> -3.8, -0.8, -1.2, -2.1, -0.7, -4.3, -12.4, -5.0, 12.0, 2.5, 7.5, 5.9...
```

Some Preprocessing

- `pres20` will be used for evaluation:

```
> pres20 <- as_tibble(read.csv(url("https://bit.ly/47b10aU")))
> pres20 <- pres20 |>
    mutate(margin = biden - trump)

> glimpse(pres20)
Rows: 51
Columns: 6
$ state  <chr> "AK", "AL", "AR", "AZ", "CA", "CO", "CT", "DC", "DE", "FL", "GA", "HI", . . .
$ ev     <int> 3, 9, 6, 11, 55, 9, 7, 3, 3, 29, 16, 4, 6, 4, 20, 11, 6, 8, 8, 11, 10, 4,. . .
$ biden  <dbl> 42.77195, 36.56999, 34.77506, 49.36469, 63.48395, 55.01107, 59.26073, 92.. . .
$ trump  <dbl> 52.833143, 62.031643, 62.395730, 49.055981, 34.320724, 41.604126, 39.1871. . .
$ other  <dbl> 0.73248408, 0.69918331, 0.25720074, 0.26322139, 0.24392550, 0.16118130, 0. . .
$ margin <dbl> -10.0611910, -25.4616530, -27.6206679, 0.3087096, 29.1632233, 13.4069415,. . .
```

Reminder of Our Goal

- Coding strategy:
    1. For each state, subset to polls within that state.
    2. Further subset the latest polls (i.e., latest information available).
    3. Average the latest polls to estimate support for each candidate.
    4. Allocate the electoral votes to the candidate who has the greatest support.
    5. Repeat this for all states and aggregate the EC votes.

Poll Prediction for Each State

- Get the list of unique state names to iterate over:

```
> poll_pred <- rep(NA, 51) # generate vector to hold the output
> state_names <- sort(unique(polls20$state))
```

- Add labels to the holder and loop:

```
# add labels to holder
> names(poll_pred) <- state_names

> for (i in 1:51) {
      state_data <- polls20 |> filter(state == state_names[i])
      latest <- state_data |> filter(days_left == min(days_left))
      poll_pred[i] <- mean(latest$margin)
  }
```

- Examine the output:

```
> head(poll_pred) # Margin of victory for Biden in Alaska, Alabama, Arkansas...

AK    AL    AR    AZ    CA    CO
-9.00 -26.00 -23.00 4.25  26.00  11.00
```

Tidyverse **Alternative Version**

- Calculate the predicted margin for Biden (vs. Trump) in each state:

```
> poll_pred <- polls20 |>
    group_by(state) |>
    filter(days_left == min(days_left)) |>
    summarize(margin_pred = mean(biden - trump)); poll_pred

# A tibble: 51 × 2
   state margin_pred
   <chr>       <dbl>
 1 AK          -9
 2 AL         -26
 3 AR         -23
 4 AZ           4.25
 5 CA          26
 6 CO          11
 7 CT          22
 8 DC          89
 9 DE          22
10 FL           0.0800
# ℹ 41 more rows
# ℹ Use `print(n = ...)` to see more rows
```

3/ Evaluating the Predictions

Polling/Prediction Errors

- **Prediction error** = actual outcome − predicted outcome

```
> poll_pred <- poll_pred |>
    left_join(pres20, by = "state") |>
    mutate(errors = margin - margin_pred); poll_pred

# A tibble: 51 × 8
   state margin_pred    ev biden trump  other  margin errors
   <chr>       <dbl> <int> <dbl> <dbl>  <dbl>   <dbl>  <dbl>
 1 AK           -9          3  42.8 52.8  0.732  -10.1   -1.06
 2 AL          -26          9  36.6 62.0  0.699  -25.5    0.538
 3 AR          -23          6  34.8 62.4  0.257  -27.6   -4.62
 4 AZ            4.25      11  49.4 49.1  0.263    0.309 -3.94
 5 CA           26         55  63.5 34.3  0.244   29.2    3.16
 6 CO           11          9  55.0 41.6  0.161   13.4    2.41
 7 CT           22          7  59.3 39.2  0.129   20.1   -1.93
 8 DC           89          3  92.1  5.40 0.491   86.8   -2.25
 9 DE           22          3  58.7 39.8  0.0780  19.0   -3.03
10 FL            0.0800    29  47.9 51.2  0.0835  -3.36  -3.44
# ℹ 41 more rows
# ℹ Use `print(n = ...)` to see more rows
```

  - A negative error means Biden actually did worse than predicted.
    - Alaska (AK): `margin` − `margin_pred` = −10.1 − (−9) = −1.06.
- Florida (FL, row 10): the polls had Biden ahead by 0.08 points.
  - Q: What was the actual margin, and who won Florida?

Assessing the Prediction Error

- **Bias**: average prediction error
  - The bias is negative: on average, the polls predicted a larger margin for Biden than the actual margin.

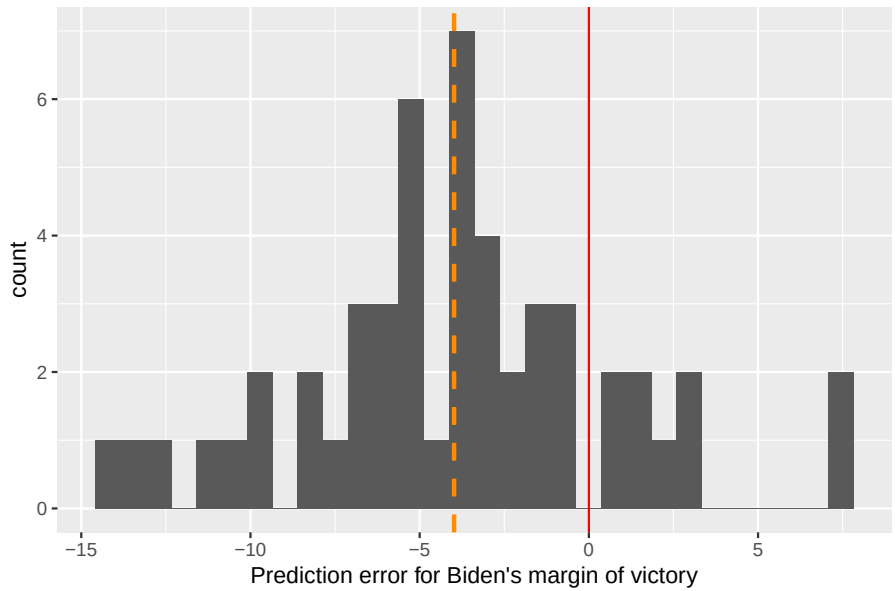
```
> mean(poll_pred$errors)
[1] -3.983248
```

- **Root mean-squared error**: average magnitude of the prediction error
  - The poll predictions were off by about 6.07 percentage points from the actual results.
  - Is 6.07 large? It depends on the context.
    - Here it is large: three states were decided by less than 1 percentage point (AZ, GA, WI).

```
> sqrt(mean(poll_pred$errors^2))
[1] 6.06975
```

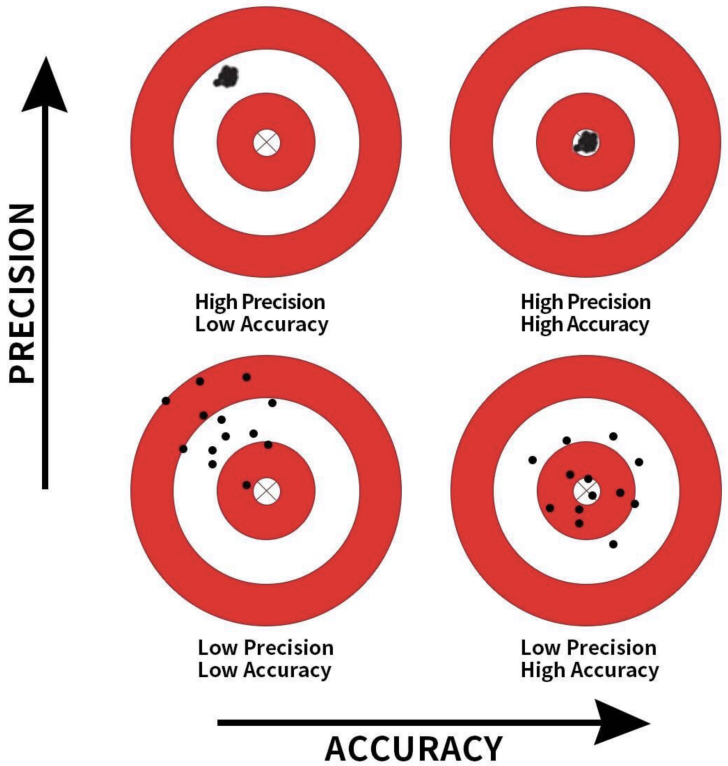
Histogram of the Errors

```
> ggplot(poll_pred, aes(x = errors)) +
    geom_histogram() +
    labs(x = "Prediction error for Biden's margin of victory") +
    geom_vline(xintercept = 0, color = "red") +
    geom_vline(xintercept = mean(poll_pred$errors),
               color = "darkorange", linetype = "dashed", linewidth = 1)
```



- Most errors are negative: the polls overestimated Biden's margin in 42 of 51 states.
  - Red line: no error (the prediction equals the actual result).
  - Orange dashed line: the bias, −3.98 (slide 24).

Two Ways to Miss: Accuracy and Precision



Source: forecast.app

- Each dot is one prediction. The center of the target is the actual result (an error of zero).
- **Accuracy**: are the dots centered on the target? Low accuracy means **bias**.
- **Precision**: are the dots close together? Low precision means a large **spread**.
- Our state polls:
  - Bias −3.98: the center of the errors (the orange dashed line on slide 25).
  - Spread 4.58: the standard deviation of the errors around that center.

RMSE Combines Bias and Spread

- Bias squared plus the variance of the errors equals RMSE squared (the mean squared error, MSE):[1]

```
> mean(poll_pred$errors)^2                          # bias squared
[1] 15.86626
> mean((poll_pred$errors - mean(poll_pred$errors))^2)   # variance (= 4.58 squared)
[1] 20.9756
> mean(poll_pred$errors^2)                          # MSE = RMSE squared
[1] 36.84187
```

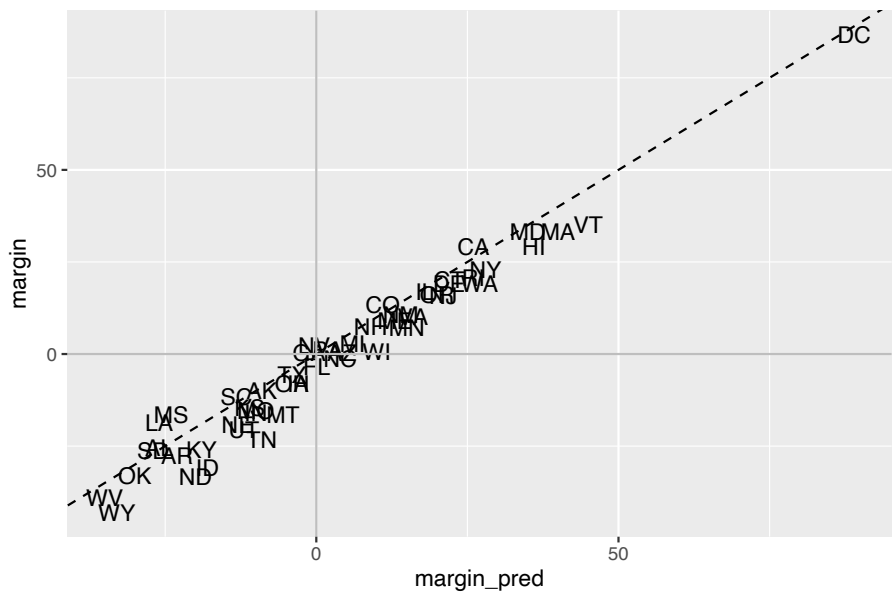
- $36.84 = 15.87 + 20.98$, so RMSE = $\sqrt{36.84} = 6.07$.
  - RMSE is the average magnitude of the prediction error (slide 24). It includes the bias, but it is not the bias.
  - Here bias squared is 43% of the MSE: the polls missed in one direction, and they also scattered.
  - In 2020, the polls overstated Biden's margin. The error was the largest in 40 years for national polls and in at least 20 years for state polls (AAPOR Task Force report, 2021).

[1] Here the variance divides by n. `var()` and `sd()` divide by $n - 1$: 21.40 and 4.63.

Comparing Polls to Outcome

- Sometimes we want to plot text labels instead of points, and we use `geom_text()` and the label aesthetic:
  - Diagonal line representing where predicted margin = actual margin.
  - e.g., Tennessee (TN) below the line: the predicted margin > actual margin of victory for Biden.

```
> ggplot(poll_pred, aes(x = margin_pred, y = margin)) +
    geom_text(aes(label = state)) +
    geom_abline(intercept = 0, slope = 1, linetype = 2) +
    geom_hline(yintercept = 0, color = "grey") +
    geom_vline(xintercept = 0, color = "grey")
```



`if_else()`: One Decision for Every Row

- `if_else(condition, true, false)` checks the condition for every element of a column at once:

```
> poll_pred |>
    mutate(winner_pred = if_else(margin_pred > 0, "Biden", "Trump")) |>
    select(state, margin_pred, winner_pred)

# A tibble: 51 × 3
   state margin_pred winner_pred
   <chr>       <dbl> <chr>
 1 AK          -9    Trump
 2 AL         -26    Trump
 3 AR         -23    Trump
 4 AZ           4.25 Biden
 5 CA          26    Biden
 6 CO          11    Biden
 7 CT          22    Biden
 8 DC          89    Biden
 9 DE          22    Biden
10 FL           0.0800 Biden
# ℹ 41 more rows
```

- We use `if_else()` again with the minimum wage data.

Classification

- Election prediction: need to predict winner in each state.
  - (Actual) number of EC votes Joe Biden won:

```
> poll_pred |>
    filter(margin > 0) |> ## States in which Biden actually won
    summarize(sum(ev)) |> pull() # pull to extract single column
[1] 306
```

  - (Predicted) number of EC votes Joe Biden won:

```
> poll_pred |>
    filter(margin_pred > 0) |> ## States in which Biden predicted to win
    summarize(sum(ev)) |> pull()
[1] 328
```

- Prediction of binary outcome variable = **classification problem**
- Wrong prediction $\rightsquigarrow$ misclassification
  1. **True positive**: predict Biden wins when he actually wins.
  2. **False positive**: predict Biden wins when he actually loses.
  3. **True negative**: predict Biden loses when he actually loses.
  4. **False negative**: predict Biden loses when he actually wins.

Classification Based on Polls

- Accuracy of our classification:
  - `sign()` returns 1 for a positive number, −1 for a negative number, and 0 for 0.

```
> poll_pred |>
    summarize(prop_correct = mean(sign(margin_pred) == sign(margin)))

# A tibble: 1 × 1
  prop_correct
         <dbl>
1        0.922
```

- Q: Which states were misclassified by the poll?:

```
> poll_pred |>
    filter(sign(margin_pred) != sign(margin))

# A tibble: 4 × 8
  state margin_pred    ev biden trump  other margin errors
  <chr>       <dbl> <int> <dbl> <dbl>  <dbl>  <dbl>  <dbl>
1 FL         0.0800    29  47.9  51.2 0.0835 -3.36   -3.44
2 GA        -1.15      16  49.5  49.2 0.0759  0.236   1.39
3 NC         3.95      15  48.6  49.9 0.296  -1.35   -5.30
4 NV        -0.350      6  50.1  47.7 0.759   2.39    2.74
```

Counting the Four Results

- Use `if_else()` twice, then `count()` each combination:

```
> poll_pred |>
    mutate(pred = if_else(margin_pred > 0, "Biden", "Trump"),
           actual = if_else(margin > 0, "Biden", "Trump")) |>
    count(pred, actual)

# A tibble: 4 × 3
  pred  actual     n
  <chr> <chr>  <int>
1 Biden Biden     24
2 Biden Trump      2
3 Trump Biden      2
4 Trump Trump     23
```

- True positives: 24. False positives: 2 (FL, NC). False negatives: 2 (GA, NV). True negatives: 23.
- Correct classifications: (24 + 23) / 51 = 0.922, the accuracy on the previous slide.

4/ Time-Series Plot

National Polls

- We often want to show a time series of the national-level polls (FiveThirtyEight) to get a sense of the popular vote.
  - Useful to bolster the credibility of our prediction!

```
> national_polls20 <- read_csv("https://bit.ly/3VOUjsa"); national_polls20

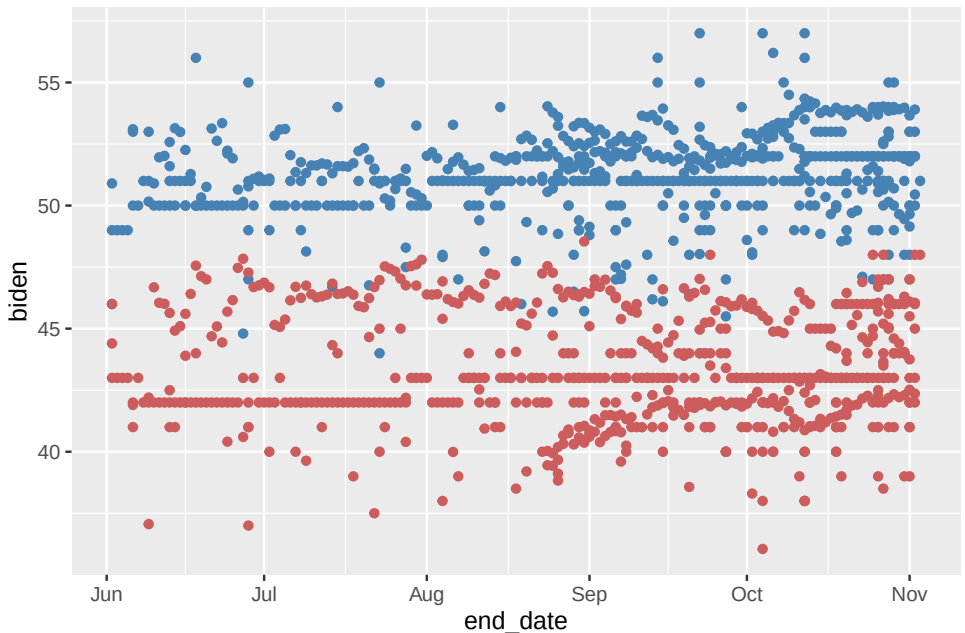
# A tibble: 654 × 5
   end_date   pollster                                sample_size biden trump
   <date>     <chr>                                         <dbl> <dbl> <dbl>
 1 2020-11-03 Lake Research                                  2400  51    48
 2 2020-11-02 Research Co.                                   1025  50    42
 3 2020-11-02 YouGov                                         1363  53    43
 4 2020-11-02 Ipsos                                           914  52    45
 5 2020-11-02 SurveyMonkey                                  28240  52    46
 6 2020-11-02 HarrisX                                        2297  52    48
 7 2020-11-02 TIPP                                           1212  50.4  46.0
 8 2020-11-02 USC Dornsife                                   5423  53.9  42.4
 9 2020-11-01 John Zogby Strategies/EMI Research Solutio.    1008  49.6  43.8
10 2020-11-01 Swayable                                       5174  51.8  46.1
# ℹ 644 more rows
# ℹ Use `print(n = ...)` to see more rows
```

Plotting the Raw Results

- Plot the polling results using `ggplot2`:

```
> national_polls20 |>
    filter(!is.na(trump)) |> # one poll has no number for Trump
    ggplot(aes(x = end_date)) +
    geom_point(aes(y = biden), color = "steelblue") +
    geom_point(aes(y = trump), color = "indianred")
```

- Fairly messy:
  - Biden is clearly ahead. But it is hard to tell whether his lead is growing or shrinking.



Clean the Mess with Moving Averages

- **Goal**: plot the average of polls in the last 7 days (challenging to do just with `dplyr`).

- Loop over each day in the data and do:
    1. Subset to all polls in the 7 days up to that day.
    2. Calculate the average of these polls for Biden and Trump.
    3. Save the result as a 1-row tibble.

Dates in R

- You can get R to properly understand dates and do arithmetic with them by storing them as `Date` objects:

```
> head(national_polls20$end_date)
[1] "2020-11-03" "2020-11-02" "2020-11-02" "2020-11-02" "2020-11-02" "2020-11-02"

> head(national_polls20$end_date + 3)
[1] "2020-11-06" "2020-11-05" "2020-11-05" "2020-11-05" "2020-11-05" "2020-11-05"
```

- We can convert a string to a date using the `lubridate` package:

```
# Don't forget to install and load the package!

> "2020-11-03" + 3  ## R doesn't know this is a date yet!
Error in "2020-11-03" + 3: non-numeric argument to binary operator

> lubridate::ymd("2020-11-03") + 3
[1] "2020-11-06"

> lubridate::mdy("11/03/2020") + 3
[1] "2020-11-06"
```

Creating a Vector of Dates

- Set up the vector of dates to cover the whole 'window':

```
> election_day <- lubridate::ymd("2020-11-03")
# + 1: start one day later, so even the first average covers two days of polls
> all_dates <- seq(from = min(national_polls20$end_date) + 1,
                   to = election_day,
                   by = "days")
> head(all_dates)
[1] "2020-06-03" "2020-06-04" "2020-06-05" "2020-06-06"
[5] "2020-06-07" "2020-06-08"
```

Moving Window Loop

- Calculate the moving averages of polls with a loop:

```
> output <- vector("list", length = length(all_dates)) # empty list to assign the output

> for (i in seq_along(all_dates)) {
    this_date <- all_dates[[i]]

    this_week <- national_polls20 |>
      filter(
        this_date - end_date >= 0, # this_date is on or after end_date
        this_date - end_date < 7   # within a week
      )

    output[[i]] <- this_week |>
      summarize(
        date = this_date,
        biden = mean(biden, na.rm = TRUE),
        trump = mean(trump, na.rm = TRUE)
      )
    }
```

- Combine all the weekly summaries into a single tibble:

```
> output <- bind_rows(output)
```

Examine the Result

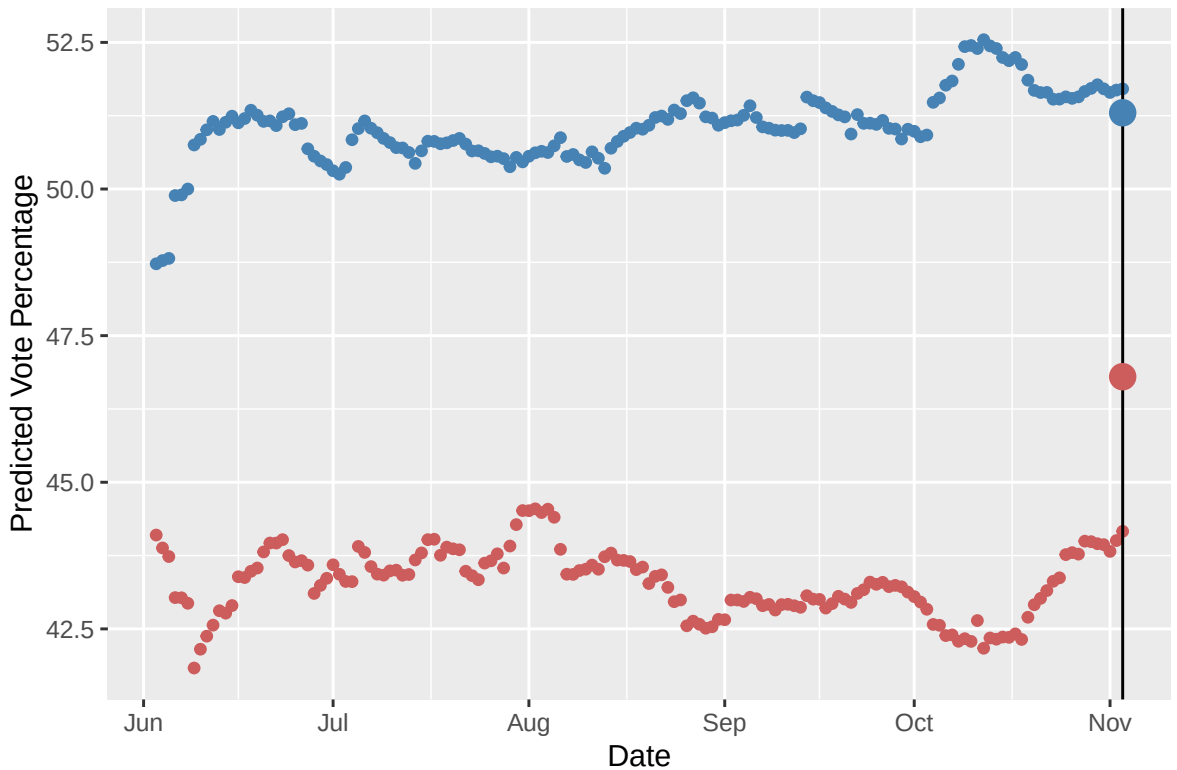
```
> output

# A tibble: 154 × 3
   date       biden trump
   <date>     <dbl> <dbl>
 1 2020-06-03  48.7  44.1
 2 2020-06-04  48.8  43.9
 3 2020-06-05  48.8  43.7
 4 2020-06-06  49.9  43.0
 5 2020-06-07  49.9  43.0
 6 2020-06-08  50    42.9
 7 2020-06-09  50.8  41.8
 8 2020-06-10  50.8  42.2
 9 2020-06-11  51.0  42.4
10 2020-06-12  51.2  42.6
# ℹ 144 more rows
# ℹ Use `print(n = ...)` to see more rows
```

- Each row is one day: the average of all polls in the 7 days up to that day.

Plotting the Moving Averages

```
> output |>
    ggplot(aes(x = date)) +
    geom_point(aes(y = biden), color = "steelblue") +
    geom_point(aes(y = trump), color = "indianred") +
    geom_vline(xintercept = election_day) +
    annotate("point", x = election_day, y = 51.3, color = "steelblue", size = 4) +
    annotate("point", x = election_day, y = 46.8, color = "indianred", size = 4) +
    labs(x = "Date", y = "Predicted Vote Percentage")
```



How Close Were the National Polls?

- The moving average on election day:

```
> output |>
    filter(date == election_day) |>
    mutate(margin_pred = biden - trump)

# A tibble: 1 × 4
  date       biden trump margin_pred
  <date>     <dbl> <dbl>       <dbl>
1 2020-11-03  51.7  44.2        7.54
```

- Actual popular vote: Biden 51.3%, Trump 46.8%
  - $\rightsquigarrow$ actual margin 4.5 percentage points.
- Prediction error = $4.5 - 7.54 \approx -3.0$: the national polls also overestimated Biden's victory, as the state polls did (bias -3.98).

5/ Review: Observational Studies

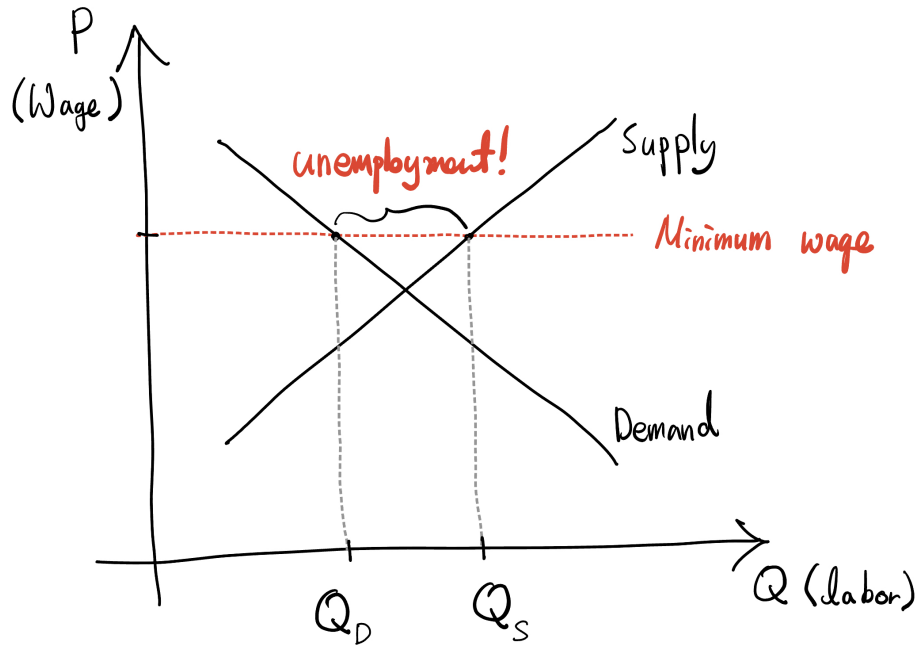
Three Questions We Can Ask with Data

- **Description**: what does the data look like?
  - Is 60% vaccinated a lot? Do two variables move together? (Week 7)
- **Prediction**: what will happen?
  - Who wins each state, from the polls (today). We judge it by the prediction error.
- **Causal inference**: what would happen if we changed something?
  - NJ employment if NJ had not raised the minimum wage (Week 4, and the next slides).
  - Causal inference is also a prediction task: we predict the counterfactual, an outcome we never observe (*counterfactual prediction*).[2]

[2] Credit: Hernán, Hsu, and Healy (2019), *CHANCE* 32(1), 42–49.

Minimum Wage Study

- **Q: Does increasing the minimum wage affect employment?**
  - Standard theory predicts that a higher minimum wage leads employers to cut jobs.
  - But that is a theoretical prediction. Can we give empirical evidence?
- Canonical study: Card and Krueger (1994) on minimum wage laws in NJ
  - In 1992, NJ raised minimum wage from \$4.25/hr to \$5.05/hr.
  - What is the effect of this change?



What Can We Do?

- **Does increasing the minimum wage affect employment?**

- Three ways to analyze data (designs from Week 4):

    - Cross-sectional: Compare NJ employment to neighbor PA employment in November 1992 (after) $\rightsquigarrow$ limited by **unmeasured confounding**.
    - Before-and-after: Compare changes in NJ employment between February 1992 (before) and November 1992 (after) $\rightsquigarrow$ limited by **time-varying confounding**.

    - **Difference-in-differences**: Compare changes in NJ employment between February 1992 (before) and November 1992 (after) to changes in PA in the same period.

Card & Krueger Data

Name	Description
chain	Name of the fast-food restaurant chain
location	Location of the restaurant
wageBefore	Wage at the restaurant before NJ minimum wage law
wageAfter	Wage at the restaurant after NJ minimum wage law
fullBefore	Number of full-time employees before NJ minimum wage law
fullAfter	Number of full-time employees after NJ minimum wage law
partBefore	Number of part-time employees before NJ minimum wage law
partAfter	Number of part-time employees after NJ minimum wage law

- Import data:

```
> minwage <- as_tibble(read.csv(url("https://bit.ly/3QZmodH"))); head(minwage)

# A tibble: 6 × 8
  chain      location wageBefore wageAfter fullBefore fullAfter partBefore partAfter
  <chr>      <chr>         <dbl>     <dbl>      <dbl>     <dbl>      <dbl>     <dbl>
1 wendys     PA              5         5.25         20         0         20        36
2 wendys     PA              5.5       4.75          6        28         26         3
3 burgerking PA              5         4.75         50        15         35        18
4 burgerking PA              5         5            10        26         17         9
5 kfc        PA              5.25      5             2         3          8        12
6 kfc        PA              5         5             2         2         10         9
```

Creating a Treatment Vector

```
> minwage <- minwage |>
    mutate(
      state = if_else(location == "PA", "PA", "NJ") ## PA is control
    ); head(minwage, 2)

# A tibble: 2 × 9
  chain  location wageBefore wageAfter fullBefore fullAfter partBefore partAfter state
  <chr>  <chr>         <dbl>     <dbl>      <dbl>     <dbl>      <dbl>     <dbl> <chr>
1 wendys PA              5        5.25         20         0         20        36 PA
2 wendys PA              5.5      4.75          6        28         26         3 PA
```

- Count the number of restaurants by location:

```
> minwage |>
    count(location)

# A tibble: 5 × 2
  location      n
  <chr>     <int>
1 PA           67
2 centralNJ    45
3 northNJ     146
4 shoreNJ      33
5 southNJ      67
```

- The paper's main estimate also combines the four NJ areas into one treated group. All PA restaurants are in eastern Pennsylvania, near NJ.

Estimating the Difference-in-Differences

```
> minwage |>
    group_by(state) |>
    mutate(fte_after = fullAfter + 0.5 * partAfter,
           fte_before = fullBefore + 0.5 * partBefore) |>
    summarize(trend = mean(fte_after) - mean(fte_before)) |>
    pivot_wider(
      names_from = state,
      values_from = trend
    ) |>
    mutate(DID = NJ - PA)

# A tibble: 1 × 3
     NJ    PA   DID
  <dbl> <dbl> <dbl>
1 0.312 -2.13  2.45
```

- **Interpretation**: the minimum wage law increased FTE employment in NJ by about 2.45 workers per restaurant. Card and Krueger (1994) report 2.76.
  - FTE (full-time equivalent) employment = full-time + 0.5 × part-time.
  - **Parallel time trend assumption**: changes in FTE employment in NJ would be roughly the same as in PA if NJ had not raised the minimum wage.
  - So 2.45 = NJ's actual change — the change we predict for NJ without the law (PA's change).

Prediction vs. Causal Inference: Which Variables?

- **Prediction**: use any variable that helps predict new cases, even if it is not a cause.
  - The number of firefighters at a fire predicts the damage well. But sending fewer firefighters would not reduce the damage.
  - More variables are not always better: a model can fit the noise in the data at hand (more in the regression weeks).

- **Causal inference**: choose the variables with a DAG (Week 6).
  - Adjust for confounders. Do not adjust for mediators or colliders.
  - A variable that predicts the outcome well can still be a bad control.

6/ In-Class Exercise E4

In-Class Exercises: Background

- We want to find out the answer to the following question:
  - Q: do we predict better if we average all polls from the last 7 days, instead of only the latest polls?
- `polls20` and `pres20` from today's class can answer this, so keep working in the same R session.
  - Some relevant variables:
    - `days_left`: number of days between the end date of the poll and election day
    - `biden`, `trump`: predicted support in the poll (percentage)
    - `margin`, `ev` in `pres20`: Biden's actual margin of victory and the electoral votes of the state

In-Class Exercises: Task

- Recreate the following table with the polls from the last 7 days:

```
# A tibble: 1 × 3
   bias  rmse accuracy
  <dbl> <dbl>    <dbl>
1 -4.40  5.96    0.961
```

- Compare it with today's results from the latest polls: bias −3.98, RMSE 6.07, accuracy 0.922.
    - To get started, we'll give you a few clues on the following slide.

In-Class Exercises: Instructions

1. Start from the `tidyverse` code on slide 22. Replace `filter(days_left == min(days_left))` with `filter(days_left <= 7)`.
2. Join `pres20` and compute `errors` as on slide 23.
3. In one `summarize()`, compute `bias`, `rmse`, and `accuracy` (slides 24 and 31).
4. Verify before you trust the output: you should get 51 states and no NA.
5. Then: (a) predict Biden's electoral votes with these polls (slide 30); (b) change the window of the moving average from 7 to 14 days (slide 38); (c) estimate the difference in differences for wages (`wageAfter`, `wageBefore`) as on slide 47.

Q: Which method predicts better? Does your answer depend on the measure (bias, RMSE, or accuracy)?

What Comes Next?

- Today: predicting the 2020 election from state polls, for loops, evaluating predictions, time-series plots with moving averages, and a review of difference-in-differences.
- Next class (Nov 5): **regression and model fit**. Complete the Required reading in the weekly schedule beforehand.
- Assignment 2 is due before class starts on Thursday, Nov 5.
- E4 (today's exercise): submit on eeclass by 23:59 today.

Enjoy Your Weekends! :)

Contact Information:
jaewon.yoo@iss.nthu.edu.tw
https://j1yoo.github.io/

